ACT / ALOHA · arXiv 2304.13705 · RSS 2023

Stakeholder

18+3

Cheap bimanual data engine
+ action-chunking control prior

Oct 12 paper club · VLA Architectures

Spine: action chunking as motor contract — TE ≠ receding ≠ open-loop ≠ soft-inpaint

Zhao, Kumar, Levine, Finn · Stanford / Berkeley / Meta

Where the value lives in 2026

Data engine

Cheap bimanual teleop that collects zip-tie / NIST / juggling demos — cultural default for ALOHA-class data

Control prior

Action chunks as motor contract. BEHAVIOR stacks inherit horizons & overlap — whether or not they still train CVAE

Not a FM

Per-task ~80M from scratch. No language.

Value = hardware + chunking recipe, not exact weights

Ask of leadership

Treat ALOHA-class teleop + action-chunk policies as data/control infrastructure, not a 2023 one-off demo. Follow-ons: Mobile ALOHA · ALOHA 2 · OpenPI/BEHAVIOR.

Do NOT redo Archaeologist's full citation dig — steal the club line once, stay on value

Numbers leadership remembers

<$20k

full bimanual system

~50 demos

~10–20 min / task

84–96%

Cup / Battery / Ziploc / Shoe

k=100

~2 s @ 50 Hz default

~80M

per-task from scratch

~5 h

train on 11 GB 2080 Ti

Low-Cost Hardware

Tony Z. Zhao[1] Vikash Kumar[3] Sergey Levine[2] Chelsea Finn[1]
[1] Stanford University [2] UC Berkeley [3] Meta

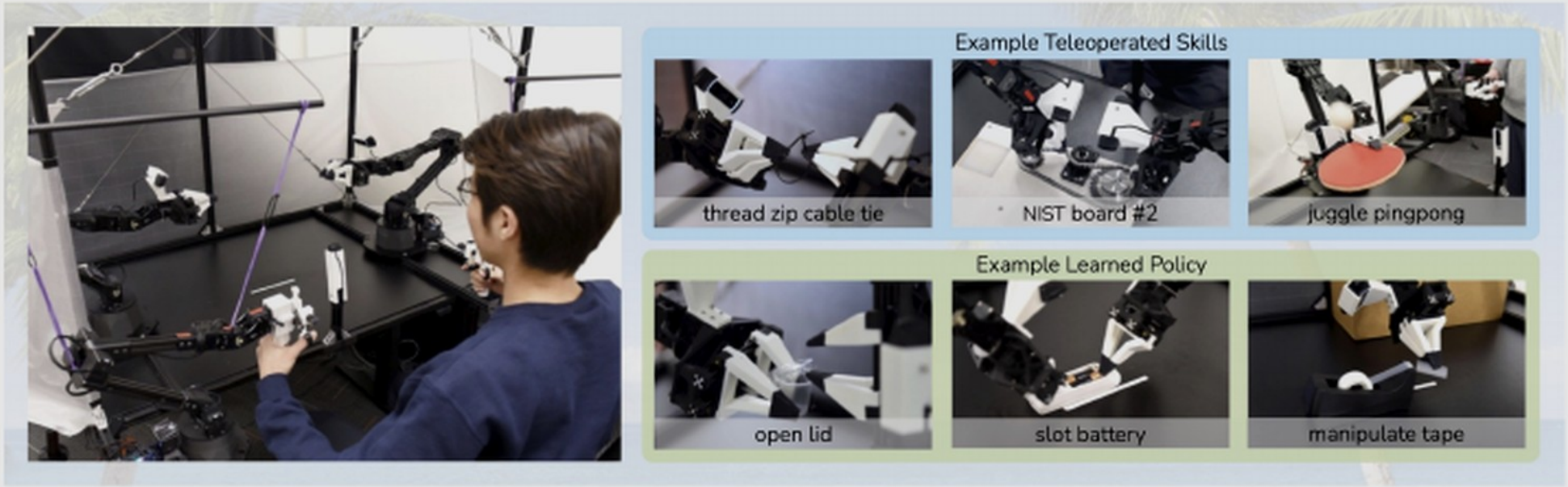


Fig. 1: *ALOHA* : *A Low-cost Open-source Hardware System for Bimanual Teleoperation*. The whole system costs <$20k with off-the-shelf robots and 3D printed components. *Left:* The user teleoperates by backdriving the leader robots, with the follower robots mirroring the motion. *Right: ALOHA* is capable of precise, contact-rich, and dynamic tasks. We show examples of both teleoperated and learned skills.

Abstract*—Fine manipulation tasks, such as threading cable ties or slotting a battery, are notoriously difficult for robots because they require precision, careful coordination of contact forces, and closed-loop visual feedback. Performing these tasks typically requires high-end robots, accurate sensors, or careful calibration, which can be expensive and difficult to set up. *Can learning enable low-cost and imprecise hardware to perform these fine

off the table. Next, one of the right fingers approaches the cup from below and pries the lid open. Each of these steps requires high precision, delicate hand-eye coordination, and rich contact. Millimeters of error would lead to task failure.

Existing systems for fine manipulation use expensive robots and high-end sensors for precise state estimation [29, 60, 32]

ALOHA hardware · <$20k leader–follower bimanual

WidowX leaders → ViperX followers · joint-space teleop · 4 RGB · 50 Hz

Cage + workspace

joint-space map

Leaders

WidowX ×2

handles + scissor

Human

Followers

ViperX 300 ×2

750 mm · 750 g

1 mm repeat · 5–8 mm acc.

top

front

wrist L

wrist R

Budget

<$20k total
2×ViperX + 2×WidowX
+ cage + cams

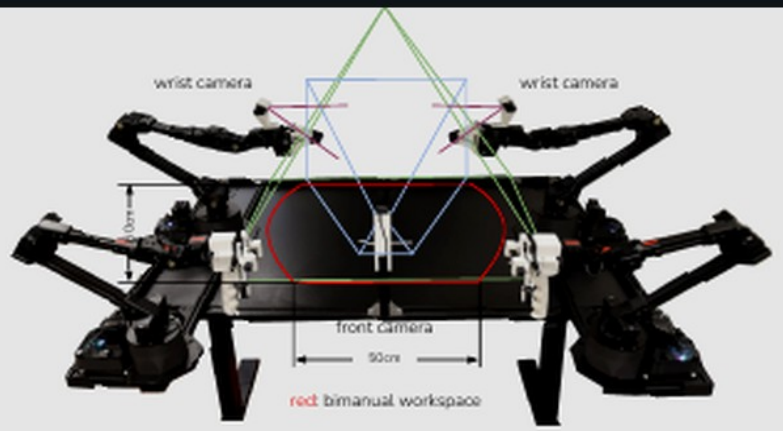
Control

50 Hz record/control
Absolute joint targets
Dynamixel PID

Design

Low-cost · versatile
Repairable · <2 h build
Rubber-band gravity assist

Follow-ons: Mobile ALOHA (2401.02117) · ALOHA 2 (2405.02292) — data engines, not just demos



ViperX 6dof Arm (follower)

#Dofs	6+gripper
Reach	750mm
Span	1500mm
Repeatability	1mm
Accuracy	5-8mm
Working Payload	750g

Fig. 3: *Left:* Camera viewpoints of the front, top, and two wrist cameras, together with an illustration of the bimanual workspace of *ALOHA*. *Middle:* Detailed view of the "handle and scissor" mechanism and custom grippers. *Right:* Technical spec of the ViperX 6dof robot [1].

motor primitives [20, 19, 50]. Some of the works also focus on fine-grained manipulation tasks such as knot untying, cloth flattening, or even threading a needle [19, 18, 31], while using robots that are considerably more expensive, e.g. the da Vinci surgical robot or ABB YuMi. Our work turns to low-cost hardware, e.g. arms that cost around $5k each, and seeks to enable them to perform high-precision, closed-loop tasks. Our teleoperation setup is most similar to Kim et al. [32], which also uses joint-space mapping between the leader and follower robots. Unlike this previous system, we do not make use of special encoders, sensors, or machined components. We build our system with only off-the-shelf robots and a handful of 3D printed parts, allowing non-experts to assemble it in less than 2 hours.

III. ALOHA: A Low-cost Open-source Hardware System for Bimanual Teleoperation

We seek to develop an accessible and high-performance

when performing delicate operations, and robust grip even with thin plastic films.

We then seek to design a teleoperation system that is maximally user-friendly around the ViperX robot. Instead of mapping the hand pose captured by a VR controller or camera to the end-effector pose of the robot, i.e. task-space mapping, we use direct joint-space mapping from a smaller robot, WidowX, manufactured by the same company and costs $3300 [2]. The user teleoperates by backdriving the smaller WidowX ("the leader"), whose joints are synchronized with the larger ViperX ("the follower"). When developing the setup, we noticed a few benefits of using joint-space mapping compared to task-space. (1) Fine manipulation often requires operating near singularities of the robot, which in our case has 6 degrees of freedom and no redundancy. Off-the-shelf inverse kinematics (IK) fails frequently in this setting. Joint space mapping, on the other hand, guarantees high-bandwidth control within the joint limits, while also requiring less computation and reducing

Real-task finals (ACT) · Table I/II

Slide Ziploc	Slot Battery	Open Cup	Put On Shoe	Prep Tape	Thread Velcro
88%	96%	84%	92%	64%	20%
final	final	final	final	final	final

Velcro cascade 92→40→20 (perception ceiling) · priors often 0% final

~50 demos/task (100 Velcro) · 1 seed × 25 real · BeT stuck at early stages

Honest: headline 80–90% = easier finals; Insertion sim still hard

Left: Cube Transfer. Transfer the red cube to the other arm. The right arm touches *(#1)* and grasps *(#2)* the red cube, then hands it to the left arm.
Right: Bimanual Insertion. Insert the red peg into the blue socket. Both arms grasp *(#1)*, let socket and peg make contact *(#2)* and insertion.

Fig. 7: *Simulated Task Definitions.* For each of the 2 simulated tasks, we illustrate the initializations and the subtasks.

	Cube Transfer (sim)			**Bimanual Insertion** (sim)			**Slide Ziploc** (real)			**Slot Battery** (real)		
	Touched	Lifted	Transfer	Grasp	Contact	Insert	Grasp	Pinch	Open	Grasp	Place	Insert
BC-ConvMLP	34 | 3	17 | 1	1 | 0	5 | 0	1 | 0	1 | 0	0	0	0	0	0	0
BeT	60 | 16	51 | 13	27 | 1	21 | 0	4 | 0	3 | 0	8	0	0	4	0	0
RT-1	44 | 4	33 | 2	2 | 0	2 | 0	0 | 0	1 | 0	4	0	0	4	0	0
VINN	13 | 17	9 | 11	3 | 0	6 | 0	1 | 0	1 | 0	28	0	0	20	0	0
ACT (Ours)	**97 | 82**	**90 | 60**	**86 | 50**	**93 | 76**	**90 | 66**	**32 | 20**	**92**	**96**	**88**	**100**	**100**	**96**

TABLE I: Success rate (%) for 2 simulated and 2 real-world tasks, comparing our method with 4 baselines. For the two simulated tasks, we report [training with scripted data | training with human data], with 3 seeds and 50 policy evaluations each. For the real-world tasks, we report training with human data, with 1 seed and 25 evaluations. Overall, ACT significantly outperforms previous methods.

	Open Cup (real)			**Thread Velcro** (real)			**Prep Tape** (real)				**Put On Shoe** (real)			
	Tip Over	Grasp	Open Lid	Lift	Grasp	Insert	Grasp	Cut	Handover	Hang	Lift	Insert	Support	Secure
BeT	12	0	0	24	0	0	8	0	0	0	12	0	0	0
ACT (Ours)	**100**	**96**	**84**	**92**	**40**	**20**	**96**	**92**	**72**	**64**	**100**	**92**	**92**	**92**

TABLE II: Success rate (%) for the remaining 3 real-world tasks. We only compare with the best performing baseline BeT.

tape dispenser. Especially from the top view, it is challenging to localize the velcro tie because of the small projected area.

the handover, and the left gripper should always move to a position that can grasp the tape, instead of trying to memorize where exactly the handover happens, which can vary across

ACT architecture · CVAE + Transformer

~80M · L1 + β KL (β=10) · k×14 absolute joints · z=0 at test

CVAE Encoder (train only)

[CLS] + joints + a_{t:t+k}

BERT-like Transformer

z ~ q_φ (dim 32)

discarded at test

z

ACT Policy Decoder (test-time path)

cam1 | cam2 | cam3 | cam4 — ResNet-18 ×4

→ 1200×512 tokens + joints + z → 1202×512

Transformer Encoder (4 layers)

Transformer Decoder (7 layers, k queries)

â_{t:t+k} ∈ R^{k × 14}

Obs: 4× RGB 480×640 + 14 joints · Action: absolute joint targets (not delta) · PID @ Dynamixel

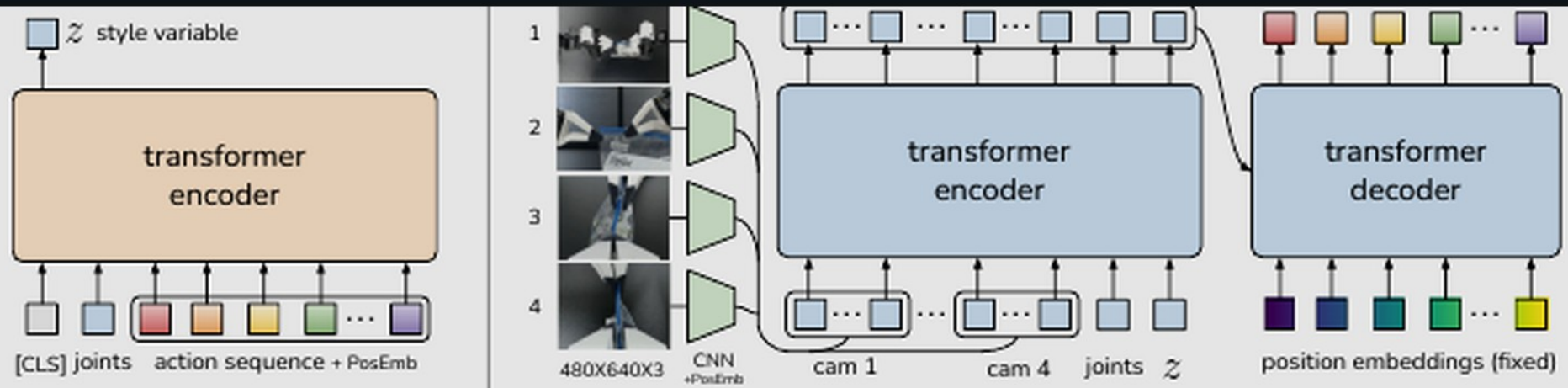


Fig. 4: *Architecture of Action Chunking with Transformers (ACT).* We train ACT as a Conditional VAE (CVAE), which has an encoder and a decoder. *Left:* The encoder of the CVAE compresses action sequence and joint observation into z, the style variable. The encoder is discarded at test time. *Right:* The decoder or policy of ACT synthesizes images from multiple viewpoints, joint positions, and z with a transformer encoder, and predicts a sequence of actions with a transformer decoder. z is simply set to the mean of the prior (i.e. zero) at test time.

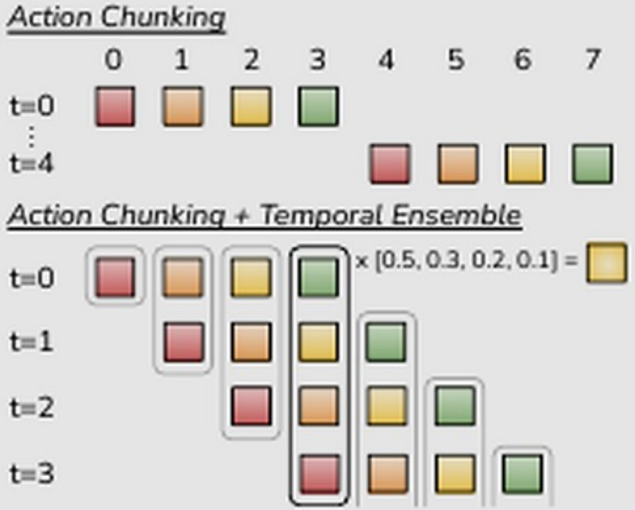


Fig. 5: We employ both Action Chunking and Temporal Ensembling when applying actions, instead of interleaving observing and executing.

To train ACT on a new task, we first collect human demonstrations using *ALOHA*. We record the joint positions of the leader robots (i.e. input from the human operator) and use them as actions. It is important to use the leader joint positions instead of the follower's, because the amount of force applied is implicitly defined by the difference between them, through the low-level PID controller. The observations are composed of the current joint positions of follower robots and the image feed from 4 cameras. Next, we train ACT to predict the *sequence of future actions* given the current observations. An action here corresponds to the target joint positions for both arms in the next time step. Intuitively, ACT tries to imitate what a human operator would do in the following time steps given current

Control schemes — do NOT conflate

Chunking is shared · TE ≠ receding ≠ open-loop ≠ soft-inpaint

Scheme	Mechanism	Commit / loop
ACT TE	Every-step query; exp-avg overlapping preds for same t	ensembled 1-step @ 50 Hz
DP receding	Predict Tp, exec Ta < Tp, replan	Ta≈8 @ ~10 Hz
π_0 open-loop	H=50; exec prefix; NO TE (hurt early for them)	open-loop prefix @ 20–50 Hz
RTC soft-inpaint	Async; freeze prefix; soft-mask remainder	latency-aware async
BEHAVIOR'25	Predict 30 / exec 26 / keep 4 + correlated FM	receding ~30 Hz

Map: k/50 ↔ DP Ta·Δt ↔ π_0/Comet H/Hz · Defaults: k=100 (~2s @ 50 Hz); L1+βKL; test z=0

Action chunking vs Temporal Ensembling

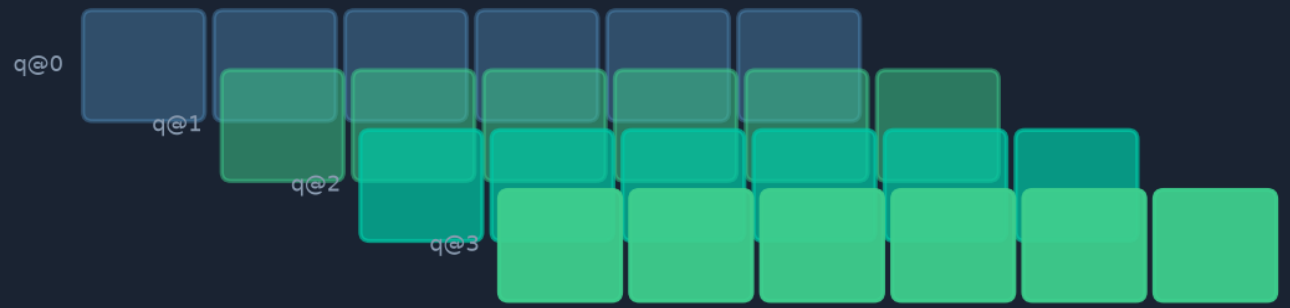
Naïve open-loop every k steps vs every-step query + exp-weighted overlap

Naïve chunking (open-loop)

observe → predict k → execute all k → replan · jerky at boundaries

ACT Temporal Ensembling

Query every tick · buffer overlapping preds for same absolute t · $a_t = \Sigma w_i \hat{A}_t[i] / \Sigma w_i \cdot w_i = \exp(-m \cdot i)$



Same-t fusion

$w = [e^{0}, e^{-m}, e^{-2m}, \ldots]$

recent obs → faster (small m)

+~3.3 pp ACT (paper)

Club line

ACT coined chunk + ensemble for 50 Hz fine bimanual BC

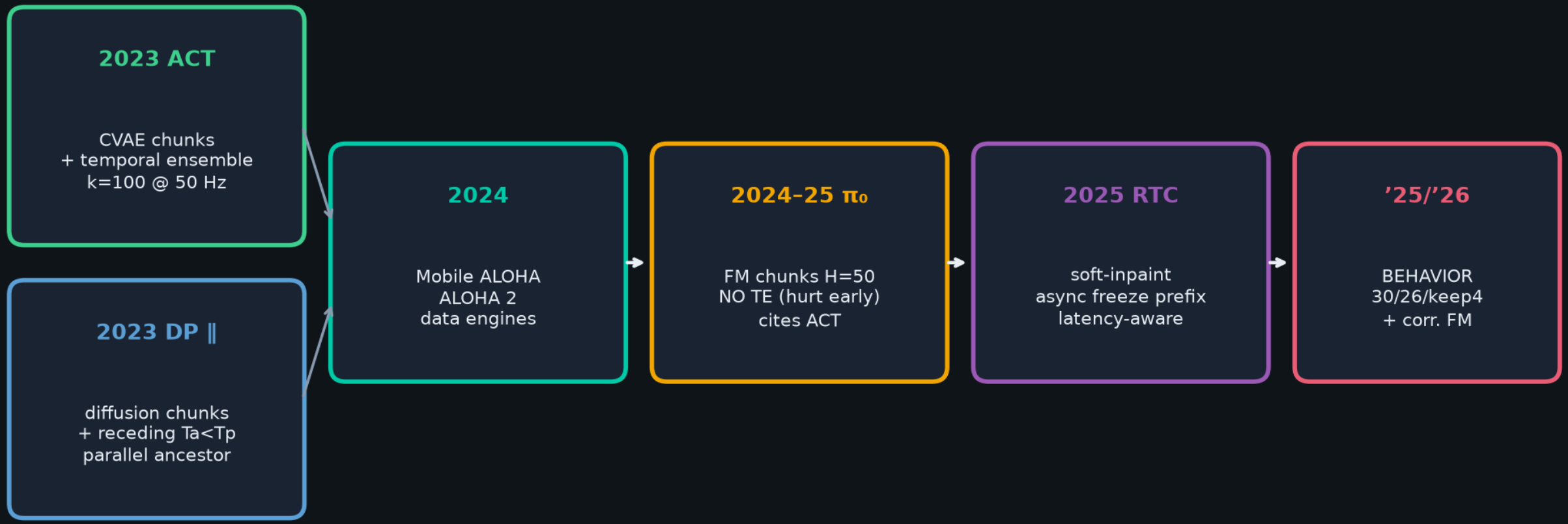
Later VLAs keep chunks, often drop ensemble,

retune H / commit Hz · TE ≠ receding ≠ open-loop ≠ soft-inpaint

Map horizons by commit_seconds — not raw k or H

Genealogy · CLEAR lineage

ALOHA+ACT (CVAE+TE) ‖ DP → Mobile/ALOHA2 → π₀/π₀.₅ → RTC → BEHAVIOR'25/'26



STAYED: generative continuous chunks · overlap / consistency at boundaries

MOVED: CVAE→diffusion→flow · TE→receding→soft-inpaint · tabletop→mobile→household VLA

Horizon map · normalize by commit seconds

ACT	k=100 @ 50 Hz	**~2.0 s chunk**	TE → 1-step ensembled
DP	Tp=16, Ta=8 @ 10 Hz	**~0.8 s commit**	receding replan
π_0	H=50 @ 20–50 Hz	**~1.0–2.5 s**	open-loop prefix
BEHAVIOR'25	30 / exec 26 @ ~30 Hz	**~0.87 s**	keep 4 + inpaint

k/50 ↔ Ta·Δt ↔ H/Hz — Empiricist + Archaeologist shared language

position embedding to the feature sequence [6]. Repeating this for all 4 images gives a feature sequence of 1200×512 in dimension. We then append two more features: the current joint positions and the "style variable" z. They are projected from their original dimensions to 512 through linear layers respectively. Thus, the input to the transformer encoder is 1202×512. The transformer decoder conditions on the encoder output through cross-attention, where the input sequence is a fixed position embedding, with dimensions $k \times 512$, and the keys and values are coming from the encoder. This gives the transformer decoder an output dimension of $k \times 512$, which is then down-projected with an MLP into $k \times 14$, corresponding to the predicted target joint positions for the next k steps. We use L1 loss for reconstruction instead of the more common L2 loss: we noted that L1 loss leads to more precise modeling of the action sequence. We also noted degraded performance when using delta joint positions as actions instead of target joint positions. We include a detailed architecture diagram in Appendix C.

We summarize the training and inference of ACT in Algorithms 1 and 2. The model has around 80M parameters, and we train from scratch for each task. The training takes around 5 hours on a single 11G RTX 2080 Ti GPU, and the inference time is around 0.01 seconds on the same machine.

V. Experiments

We present experiments to evaluate ACT's performance on fine manipulation tasks. For ease of reproducibility, we build two simulated fine manipulation tasks in MuJoCo [63], in addition to 6 real-world tasks with *ALOHA*. We provide videos for each task on the project website.

A. Tasks

All 8 tasks require fine-grained, bimanual manipulation, and are illustrated in Figure 6. For ***Slide Ziploc***, the right gripper needs to accurately grasp the slider of the ziploc bag and open it, with the left gripper securing the body of the bag. For ***Slot Battery***, the right gripper needs to first place the battery into the slot of the remote controller, then using the tip of fingers to delicately push in the edge of the battery, until it is fully inserted. Because the spring inside the battery slot causes the remote controller to move in the opposite direction during insertion, the left gripper pushes down on the remote to keep

table, followed by the right gripper pinching the tail of the tie in mid-air. Then, both arms coordinate to insert one end of the velcro tie into the other in mid-air. The loop measures 3mm x 25mm, while the velcro tie measures 2mm x 10-25mm depending on the position. For this task to be successful, the robot must use visual feedback to correct for perturbations with each grasp, as even a few millimeters of error during the first grasp will compound in the second grasp mid-air, giving more than a 10mm deviation in the insertion phase. For ***Prep Tape***, the goal is to hang a small segment of the tape on the edge of a cardboard box. The right gripper first grasps the tape and cuts it with the tape dispenser's blade, and then hands the tape segment to the left gripper mid-air. Next, both arms approach the box, the left arm gently lays the tape segment on the box surface, and the right fingers push down on the tape to prevent slipping, followed by the left arm opening its gripper to release the tape. Similar to *Thread Velcro*, this task requires multiple steps of delicate coordination between the two arms. For ***Put On Shoe***, the goal is to put the shoe on a fixed manniquin foot, and secure it with the shoe's velcro strap. The arms would first need to grasp the tongue and collar of the shoe respectively, lift it up and approach the foot. Putting the shoe on is challenging because of the tight fitting: the arms would need to coordinate carefully to nudge the foot in, and both grasps need to be robust enough to counteract the friction between the sock and shoe. Then, the left arm goes around to the bottom of the shoe to support it from dropping, followed by the right arm flipping the velcro strap and pressing it against the shoe to secure. The task is only considered successful if the shoe clings to the foot after both arms releases. For the simulated task ***Transfer Cube***, the right arm needs to first pick up the red cube lying on the table, then place it inside the gripper of the other arm. Due to the small clearance between the cube and the left gripper (around 1cm), small errors could result in collisions and task failure. For the simulated task ***Bimanual Insertion***, the left and right arms need to pick up the socket and peg respectively, and then insert in mid-air so the peg touches the "pins" inside the socket. The clearance is around 5mm in the insertion phase. For all 8 tasks, the initial placement of the objects is either varied randomly along the 15cm white reference line (real-world tasks), or uniformly in 2D regions (simulated tasks). We provide illustrations of both the initial positions and the subtasks in Figure 6 and 7. Our

Infrastructure, not a 2023 one-off demo

Cheap teleop data engine + chunking prior

Follow-ons: Mobile ALOHA · ALOHA 2 · BEHAVIOR inherits horizons/overlap

Q&A · keep the spine clear

Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware

Tony Z. Zhao[1] Vikash Kumar[3] Sergey Levine[2] Chelsea Finn[1]
[1] Stanford University [2] UC Berkeley [3] Meta

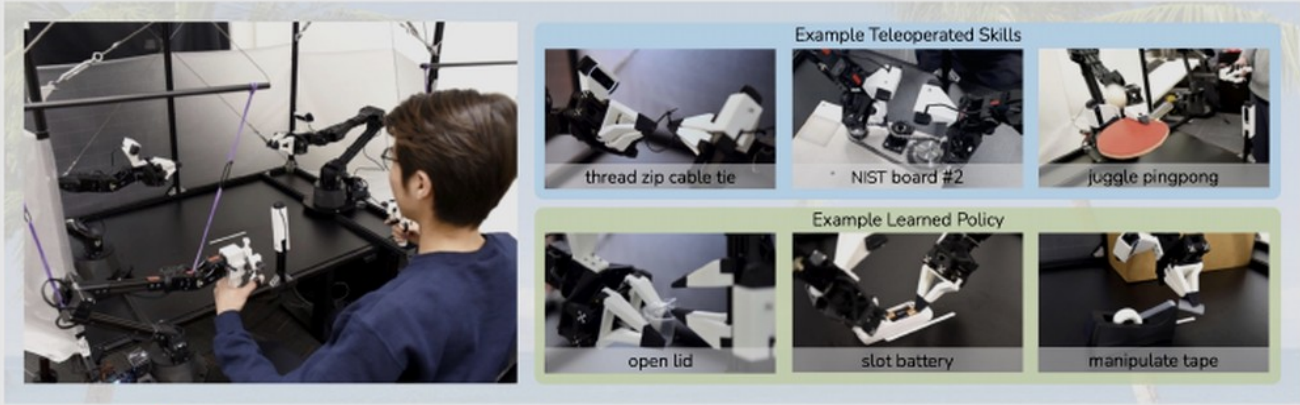


Fig. 1: *ALOHA : A Low-cost Open-source Hardware System for Bimanual Teleoperation*. The whole system costs <$20k with off-the-shelf robots and 3D printed components. *Left:* The user teleoperates by backdriving the leader robots, with the follower robots mirroring the motion. *Right: ALOHA* is capable of precise, contact-rich, and dynamic tasks. We show examples of both teleoperated and learned skills.

***Abstract*—Fine manipulation tasks, such as threading cable ties or slotting a battery, are notoriously difficult for robots because they require precision, careful coordination of contact forces, and closed-loop visual feedback. Performing these tasks typically requires high-end robots, accurate sensors, or careful calibration, which can be expensive and difficult to set up. *Can learning enable low-cost and imprecise hardware to perform these fine manipulation tasks?* We present a low-cost system that performs end-to-end imitation learning directly from real demonstrations, collected with a custom teleoperation interface. Imitation learning, however, presents its own challenges, particularly in high-precision domains: errors in the policy can compound over time, and human demonstrations can be non-stationary. To address these challenges, we develop a simple yet novel algorithm, *Action Chunking with Transformers (ACT)*, which learns a generative model over action sequences. ACT allows the robot to learn 6 difficult tasks in the real world, such as opening a translucent condiment cup and slotting a battery with 80-90% success, with only 10 minutes worth of demonstrations. Project website: tonyzhaozh.github.io/aloha**

off the table. Next, one of the right fingers approaches the cup from below and pries the lid open. Each of these steps requires high precision, delicate hand-eye coordination, and rich contact. Millimeters of error would lead to task failure.

Existing systems for fine manipulation use expensive robots and high-end sensors for precise state estimation [29, 60, 32, 41]. In this work, we seek to develop a low-cost system for fine manipulation that is, in contrast, accessible and reproducible. However, low-cost hardware is inevitably less precise than high-end platforms, making the sensing and planning challenge more pronounced. One promising direction to resolve this is to incorporate learning into the system. Humans also do not have industrial-grade proprioception [71], and yet we are able to perform delicate tasks by learning from closed-loop visual feedback and actively compensating for errors. In our system, we therefore train an end-to-end policy that directly maps RGB images from commodity web cameras to the actions.